

2. Implement matrix multiplication of two  $100 \times 100$  matrices using multithreading in Python, where each individual multiplication operation is executed by a separate thread. Synchronize the threads to compute the final resultant matrix and demonstrate the working of the algorithm using an HTML-based cloud animation.

Code:

```
import threading
import random
import time
```

```
N = 100
```

```
left = [[random.randint(0, 9) for _ in range(N)] for _ in range(N)]
right = [[random.randint(0, 9) for _ in range(N)] for _ in range(N)]
answer = [[0] * N for _ in range(N)]
```

```
row_guard = [threading.Lock() for _ in range(N)]
thread_total = 0
```

```
def product_task(r, c, step):
    part = left[r][step] * right[step][c]
    with row_guard[r]:
        answer[r][c] += part
```

```
def compute_row(r):
    """Launch N*N threads for row r, then wait for all of them."""
    global thread_total
    workers = []
    for c in range(N):
        for step in range(N):
            w = threading.Thread(target=product_task, args=(r, c, step))
            w.start()
            workers.append(w)
    for w in workers:
        w.join()
    thread_total += len(workers)
```

```
def check_result():
    """Plain single-threaded multiplication used only to verify."""
    for r in range(N):
        for c in range(N):
            expected = sum(left[r][k] * right[k][c] for k in range(N))
            if expected != answer[r][c]:
                return False
```

```

return True

if __name__ == "__main__":
    t0 = time.perf_counter()
    for r in range(N):
        compute_row(r)
        if (r + 1) % 20 == 0:
            print(f"rows finished: {r + 1}/{N}")
    elapsed = time.perf_counter() - t0

    print("\nMatrix size      :", f"{N} x {N}")
    print("Threads created   :", thread_total)
    print("Time taken        :", f"{elapsed:.2f} s")
    print("answer[0][0..4]    :", answer[0][:5])
    print("Result verified    :", "correct" if check_result() else "WRONG")

```

### Animation part:

```

<!DOCTYPE html>
<html lang = "en">
<head>
<meta charset = "UTF-8">
<meta name = "viewport" content = "width = device-width, initial-scale = 1.0">
<title>Threaded Matrix Multiplication - Cloud View</title>
<style>
:root {
  --sky-top:
  --sky-low:
  --ink:
  --sun:
  --rain:
  --cloud:
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  font-family: "Trebuchet MS", "Segoe UI", sans-serif;
  color: var(--ink);
  background: linear-gradient(var(--sky-top), var(--sky-low) 70%);
}
header { max-width: 900px; margin: 0 auto; padding: 24px 20px 0; }

```

```

h1 { margin: 0 0 6px; font-size: 1.7rem; }
header p { margin: 0; max-width: 62ch; line-height: 1.5; }

.panel { display: flex; flex-wrap: wrap; gap: 12px; align-items: center;
  max-width: 900px; margin: 16px auto 0; padding: 0 20px; }
button { font: inherit; padding: 8px 18px; border-radius: 20px; cursor: pointer;
  border: 2px solid var(--ink); background: var(--cloud); color: var(--ink); }
button:focus-visible, input:focus-visible { outline: 3px solid var(--sun); outline
offset: 2px; }
.stats { margin-left: auto; font-size: 0.95rem; }
.stats b { font-variant-numeric: tabular-nums; }

60px;
  display: flex; justify-content: center; align-items: center; gap: 28px; flex
wrap: wrap; }
.block h2 { margin: 0 0 8px; font-size: 1rem; text-align: center; }
.grid { display: grid; grid-template-columns: repeat(4, 52px); gap: 6px; }
.num { height: 52px; display: grid; place-items: center; font-weight: bold;
  background: rgba(255, 255, 255, 0.75); border: 2px solid var(--rain); border-radius:
8px;
  transition: background 0.2s, transform 0.2s; }
.num.pick { background: var(--sun); transform: scale(1.08); }
.num.locked { background:
.num.done { border-color: var(--ink); }
.sign { font-size: 2rem; }

.puff { position: absolute; left: 0; top: 0; padding: 6px 12px; font-size: 0.8rem;
  white-space: nowrap; background: var(--cloud); border-radius: 30px; pointer
events: none;
  box-shadow: 14px -8px 0 -4px var(--cloud), -12px -6px 0 -6px var(--cloud);
  transition: transform 1.1s ease-in-out, opacity 0.3s; z-index: 5; }
.puff.waiting { outline: 2px dashed var(--rain); }

.drift { position: fixed; top: 0; width: 140px; height: 46px; border-radius: 40px;
  background: rgba(255, 255, 255, 0.55); z-index: -1; animation: sail linear
infinite; }
.drift::after { content: ""; position: absolute; width: 60px; height: 60px; left: 30px;
top: -28px;
  border-radius: 50%; background: inherit; }
@keyframes sail { from { transform: translateX(-200px); } to { transform:
translateX(110vw); } }

height: 1.6; }

```

```

@media (prefers-reduced-motion: reduce) {
  .drift { animation: none; }
  .puff { transition: none; }
}
</style>
</head>
<body>

<div class = "drift" style = "top:70px; animation-duration:38s;"></div>
<div class = "drift" style = "top:230px; animation-duration:52s; animation-delay:-20s;"></div>
<div class = "drift" style = "top:420px; animation-duration:45s; animation-delay:-9s;"></div>

<header>
  <h1>Matrix multiplication with one thread per product</h1>
  <p>The Python program multiplies two 100 × 100 matrices using 1, 000, 000 threads.
    This page shows the same idea on 4 × 4 matrices. Each small cloud is a thread:
    it multiplies one pair  $A[i][k] \times B[k][j]$ , flies to cell  $C[i][j]$ , waits if that
    cell is locked by another thread, and then adds its product.</p>
</header>

<div class = "panel">
  <button id = "go">Start</button>
  <button id = "again">Reset</button>
  <label>Speed <input id = "speed" type = "range" min = "1" max = "5" value = "3"></label>
  <div class = "stats">
    Threads started <b id = "sStart">0</b> &nbsp; waiting <b id = "sWait">0</b> &nbsp; finished
    <b id = "sDone">0</b> / 64
  </div>
</div>

<div id = "stage">
  <div class = "block"><h2>Matrix A</h2><div class = "grid" id = "gA"></div></div>
  <div class = "sign">×</div>
  <div class = "block"><h2>Matrix B</h2><div class = "grid" id = "gB"></div></div>
  <div class = "sign">=</div>
  <div class = "block"><h2>Result C</h2><div class = "grid" id = "gC"></div></div>
</div>

<div id = "log">Press Start to launch the threads.</div>

<script>
  const N = 4;
  const stage = document.getElementById("stage");
  let hits, A, B, C, cellsA, cellsB, cellsC, lockBusy, jobs, timer, started, waiting,

```

finished, running;

```
function buildGrid(id, values) {
  const box = document.getElementById(id);
  box.innerHTML = "";
  const cells = [];
  for (let i = 0; i < N; i++) {
    cells.push([]);
    for (let j = 0; j < N; j++) {
      const d = document.createElement("div");
      d.className = "num";
      d.textContent = values[i][j];
      box.appendChild(d);
      cells[i].push(d);
    }
  }
  return cells;
}
```

```
function randomMatrix() {
  return Array.from({ length: N }, () => Array.from({ length: N }, () =>
Math.floor(Math.random() * 9) + 1));
}
```

```
function spot(el) {
  const s = stage.getBoundingClientRect(), r = el.getBoundingClientRect();
  return { x: r.left - s.left + r.width / 2, y: r.top - s.top + r.height / 2 };
}
```

```
function refresh() {
  document.getElementById("sStart").textContent = started;
  document.getElementById("sWait").textContent = waiting;
  document.getElementById("sDone").textContent = finished;
}
```

```
function delay() { return 700 / Number(document.getElementById("speed").value); }
```

```
function setup() {
  clearInterval(timer);
  document.querySelectorAll(".puff").forEach(p => p.remove());
  A = randomMatrix(); B = randomMatrix();
  C = Array.from({ length: N }, () => Array(N).fill(0));
  cellsA = buildGrid("gA", A); cellsB = buildGrid("gB", B); cellsC = buildGrid("gC", C);
  hits = Array.from({ length: N }, () => Array(N).fill(0));
}
```

```

lockBusy = Array.from({ length: N }, () => Array(N).fill(false));
jobs = [];
for (let i = 0; i < N; i++) for (let j = 0; j < N; j++) for (let k = 0; k < N; k++)
jobs.push([i, j, k]);
  started = waiting = finished = 0; running = false;
  document.getElementById("go").textContent = "Start";
  document.getElementById("log").textContent = "Press Start to launch the threads.";
  refresh();
}

```

```

function launchThread([i, j, k]) {
  started++; refresh();
  const a = cellsA[i][k], b = cellsB[k][j], target = cellsC[i][j];
  a.classList.add("pick"); b.classList.add("pick");
  setTimeout(() => { a.classList.remove("pick"); b.classList.remove("pick"); }, 450);

```

```

  const p = A[i][k] * B[k][j];
  const puff = document.createElement("div");
  puff.className = "puff";
  puff.textContent = `T${started}: ${A[i][k]}×${B[k][j]} = ${p}`;
  stage.appendChild(puff);

```

```

  const from = spot(b), to = spot(target);
  puff.style.transition = "none";
  puff.style.transform = `translate(${from.x - 40}px, ${from.y - 50}px)`;
  puff.getBoundingClientRect();
  puff.style.transition = "";
  puff.style.transform = `translate(${to.x - 40}px, ${to.y - 62}px)`;
  setTimeout(() => tryAdd(i, j, p, puff, false), 1150);
}

```

```

function tryAdd(i, j, p, puff, wasWaiting) {
  if (lockBusy[i][j]) {
    if (!wasWaiting) { waiting++; puff.classList.add("waiting"); refresh(); }
    setTimeout(() => tryAdd(i, j, p, puff, true), 120);
    return;
  }
  if (wasWaiting) { waiting--; puff.classList.remove("waiting"); }
  lockBusy[i][j] = true;
  const cell = cellsC[i][j];
  cell.classList.add("locked");
  C[i][j] += p;
  cell.textContent = C[i][j];
  setTimeout(() => {

```

```

    lockBusy[i][j] = false;
    cell.classList.remove("locked");
    puff.style.opacity = 0;
    setTimeout(() => puff.remove(), 300);
    finished++; refresh();
    if (++hits[i][j] === N) cell.classList.add("done");
    if (finished === N * N * N) {
        document.getElementById("log").textContent =
            "All 64 threads joined. Every C[i][j] now holds the sum of its 4 products, same
as the Python result for a 4 × 4 case.";
        running = false;
        document.getElementById("go").textContent = "Start";
    }
}, 250);
}

function tick() {
    if (!jobs.length) { clearInterval(timer); return; }
    launchThread(jobs.shift());
    document.getElementById("log").textContent =
        "Threads are running in parallel; dashed clouds are blocked on a locked cell.";
}

document.getElementById("go").addEventListener("click", () => {
    if (finished === N * N * N) setup();
    running = !running;
    document.getElementById("go").textContent = running ? "Pause" : "Resume";
    clearInterval(timer);
    if (running) timer = setInterval(tick, delay());
});
document.getElementById("speed").addEventListener("input", () => {
    if (running) { clearInterval(timer); timer = setInterval(tick, delay()); }
});
document.getElementById("again").addEventListener("click", setup);

setup();
</script>
</body>
</html>

```

**Overall output with animation:**

Matrix multiplication with one thread per product

The Python program multiplies two  $100 \times 100$  matrices using 1,000,000 threads. This page shows the same idea on  $4 \times 4$  matrices. Each small cloud is a thread: it multiplies one pair  $A[i][k] \times B[k][j]$ , flies to cell  $C[i][j]$ , waits if that cell is locked by another thread, and then adds its product.

Start Reset Speed

Threads started 0 waiting 0 finished 0 / 64

| Matrix A |   |   |   |          | Matrix B |   |   |   |   | Result C |   |   |   |
|----------|---|---|---|----------|----------|---|---|---|---|----------|---|---|---|
| 8        | 1 | 3 | 1 |          | 9        | 6 | 8 | 3 | = | 0        | 0 | 0 | 0 |
| 5        | 4 | 7 | 1 | $\times$ | 5        | 1 | 6 | 2 |   | 0        | 0 | 0 | 0 |
| 2        | 9 | 8 | 4 |          | 9        | 5 | 8 | 2 |   | 0        | 0 | 0 | 0 |
| 7        | 2 | 4 | 8 |          | 9        | 3 | 2 | 6 |   | 0        | 0 | 0 | 0 |

Press Start to launch the threads.

Matrix multiplication with one thread per product

The Python program multiplies two  $100 \times 100$  matrices using 1,000,000 threads. This page shows the same idea on  $4 \times 4$  matrices. Each small cloud is a thread: it multiplies one pair  $A[i][k] \times B[k][j]$ , flies to cell  $C[i][j]$ , waits if that cell is locked by another thread, and then adds its product.

Pause Reset Speed

Threads started 32 waiting 1 finished 26 / 64

| Matrix A |   |   |   |          | Matrix B |   |   |   |  | Result C |    |    |   |
|----------|---|---|---|----------|----------|---|---|---|--|----------|----|----|---|
| 8        | 1 | 3 | 1 |          | 9        | 6 | 8 | 3 |  | 113      | 72 | 64 | 0 |
| 5        | 4 | 7 | 1 | $\times$ | 5        | 1 | 6 | 2 |  | 0        | 0  | 0  | 0 |
| 2        | 9 | 8 | 4 |          | 9        | 5 | 8 | 2 |  | 0        | 0  | 0  | 0 |
| 7        | 2 | 4 | 8 |          | 9        | 3 | 2 | 6 |  | 0        | 0  | 0  | 0 |

Threads are running in parallel; dashed clouds are blocked on a locked cell.

Annotations: T30:  $4 \times 2 = 8$ , T29:  $5 \times 3 = 15$ , T31:  $7 \times 2 = 14$ , T32:  $1 \times 6 = 6$